m1

Undo/Redo Systems in Terrain Editing: History Stacks, State Management, and Delta-Based Memory Optimization

The Core Concept: The History Stack

At the heart of any Undo/Redo system lies a pair of stacks operating on a Last-In, First-Out principle. In terrain sculpting systems, every brush interaction modifies a large vertex buffer, and instead of tracking individual actions, the system captures snapshots of the entire height state at specific moments.

These snapshots represent complete terrain states rather than incremental operations. This approach simplifies restoration because reverting to a previous state only requires replacing the current buffer with a stored version.

However, this introduces a major constraint: memory consumption. A grid of 200 × 200 vertices contains 40,000 floating-point values. Since each Float32 value requires 4 bytes, a single snapshot already consumes approximately 160 KB of memory. Multiple undo steps can quickly scale this into several megabytes, making efficient storage critical.

https://www.udemy.com/course/web-based-3d-terrain-industrial-printing-engine-mastery/

Snapshot Strategy and Deep Copying

A key requirement in this system is that each stored state must be independent from the live terrain buffer. Simply pushing the reference of the current array into a stack is not sufficient, because all entries would point to the same evolving memory.

Instead, a deep copy must be created at each save point:

This ensures that the snapshot preserves the exact state of the terrain at that moment in time, unaffected by future modifications.

In practice, snapshots are usually taken when the user completes a stroke (for example, on mouse release), reducing unnecessary memory overhead while still preserving meaningful editing states.

Trainee Assignments: Technical Implementation Tasks

Task 1: Building the Stack Logic

The system requires two global structures:

- undoStack
- redoStack

The goal is to implement a saveState() function that pushes a deep copy of the current height buffer into the undoStack.

A critical design rule must be enforced: whenever a new action is performed after an undo, the redoStack must be cleared. This is necessary because the moment a new action is introduced, the previous forward history becomes invalid, breaking the linear timeline of states.

This maintains a strict chronological chain of terrain states rather than a branching history tree.

Task 2: Implementing the Trigger System

The saveState() function must be strategically bound to user input events. Typically, this is done on mouse interaction events such as onmousedown or onmouseup, depending on whether the system prioritizes pre-action or post-action capture.

The undo() function should:

- Pop the last state from undoStack
- Push the current state into redoStack
- Replace the active height buffer with the popped state
- Call updateMesh() to reflect changes in geometry

Assessment Questions: State Management

Question 1: Memory Optimization

A 250 × 250 grid contains 62,500 vertices. Each Float32 value occupies 4 bytes, meaning a single snapshot requires approximately 250 KB of memory.

If the undoStack stores 100 snapshots, the total memory usage becomes:

- 250 KB × 100 = 25,000 KB
- Approximately 24.4 MB of RAM

This demonstrates how quickly memory scales in state-based systems, especially in high-resolution terrain editors.

Question 2: State vs Command Pattern

The State Pattern stores full snapshots of the terrain, while the Command Pattern stores only the operations (brush position, radius, strength) and replays them to reconstruct the scene.

The Command Pattern becomes more reliable in scenarios where terrain resolution changes dynamically, such as V-Kafes (vertex density adjustments), because it is resolution-independent. Replaying commands adapts naturally to new geometry structures, whereas full state snapshots become incompatible when vertex counts change.

Question 3: The Ghosting Effect

If undo only restores terrain geometry but leaves environmental variables unchanged (such as sea level, lighting, or fog parameters), the system is incomplete.

True consistency requires that all dependent simulation parameters are included in the history system. Otherwise, visual mismatch occurs between terrain state and environmental state, leading to temporal artifacts known as "ghosting."

In advanced systems, not only geometry but also scene parameters should be part of the undo stack.

Optimization

Full mesh snapshots are expensive because they store all vertex data regardless of change frequency. In practice, most brush strokes only affect a small subset of the terrain.

A more efficient approach is Delta Storage, where only modified vertices are tracked.

Instead of storing the entire Float32Array, the system records:

- Vertex indices that changed
- Their original values before modification

This creates a compact change log rather than a full state copy.

Memory Efficiency Comparison

For a small brush stroke affecting, for example, 2,000 vertices out of 62,500:

- Full snapshot: ~250 KB
- Delta storage: (2,000 indices + values) ≈ ~16 KB or less

This results in significant memory savings, especially for long sculpting sessions with frequent small edits.

Final Insight

Undo/Redo systems in terrain engines are fundamentally about balancing memory cost against temporal accuracy. While full state snapshots provide simplicity and reliability, delta-based systems and command replay architectures offer scalability and adaptability for high-resolution, long-duration interactive simulations.

m2
m3

© 2026 Okan Kaplan – MIT License Read full license